

AIM2- Preparations

DAY 7 — Python OOP

Today's goal

By the end of Day 7, you should understand:

1. Classes
2. Objects
3. `__init__()`
4. `self`
5. Instance variables
6. Instance methods
7. Class attributes — basic understanding
8. Encapsulation — Python's approach
9. Inheritance
10. Method overriding
11. `super()`
12. How Python OOP differs from Java

And then we'll build a small **Student Management System**.

1. Class and Object

You already know this concept from Java.

Java

```
class Student {  
    String name;  
    int age;  
}
```

Python:

```
class Student:  
    pass
```

That's a class.

Now create an object:

```
student1 = Student()  
student2 = Student()
```

Think:

Student
↓
Class / Blueprint
↓

student1
student2

Objects

2. `__init__()` — Python constructor

In **Java** you might write:

```
public Student(String name, int age) {  
    this.name = name;  
    this.age = age;  
}
```

Python:

```
class Student:

    def __init__(self, name, age):
        self.name = name
        self.age = age
```

Then:

```
student1 = Student("Krishna", 28)
student2 = Student("Rahul", 25)
```

And:

```
print(student1.name)
print(student1.age)

print(student2.name)
print(student2.age)
```

Output:

```
Krishna
28
Rahul
25
```

3. What is self?

This is probably the **most important Python OOP concept for you today**.

In Java:

```
this.name = name;
```

Python:

```
self.name = name
```

They serve a similar purpose.

self means:

the current object

For:

```
student1 = Student("Krishna", 28)
```

inside the object, **self** refers to **student1**.

For:

```
student2 = Student("Rahul", 25)
```

self refers to **student2**.

So:

```
self.name = name
```

means:

Store this student's name inside this particular object.

4. Instance Methods

Let's make the class more useful.

```
class Student:

    def __init__(self, name, age):

        self.name = name
        self.age = age
```

```
def introduce(self):  
    print(f"Hello, my name is {self.name}.")  
    print(f"I am {self.age} years old.")
```

Create an object:

```
student = Student("Krishna", 28)
```

Call the method:

```
student.introduce()
```

Output:

```
Hello, my name is Krishna.  
I am 28 years old.
```

Notice:

```
def introduce(self):
```

We don't pass student manually.

Python automatically passes the object when you do:

```
student.introduce()
```

5. Adding Student Scores

Now let's connect this with everything you've been learning.

```
class Student:  
  
    def __init__(self, name, math, python, ai):  
        self.name = name  
        self.math = math
```

```
        self.python = python
        self.ai = ai

    def calculate_total(self):
        return self.math + self.python + self.ai

    def calculate_average(self):
        return self.calculate_total() / 3
```

Create:

```
student = Student("Krishna", 95, 88, 65)
```

Then:

```
print(student.name)
print(student.calculate_total())
print(student.calculate_average())
```

Output:

```
Krishna
248
82.66666666666667
```

Notice how nice this is.

Instead of:

```
calculate_total(math, python, ai)
```

we can do:

```
student.calculate_total()
```

because the student already contains:

```
name
math
python
ai
```

This is one of the reasons we use OOP.

6. Add Grade and Result

Let's extend it:

```
class Student:

    def __init__(self, name, math, python, ai):
        self.name = name
        self.math = math
        self.python = python
        self.ai = ai

    def calculate_total(self):
        return self.math + self.python + self.ai

    def calculate_average(self):
        return self.calculate_total() / 3

    def calculate_grade(self):
        average = self.calculate_average()

        if average >= 90:
            return "A"
        elif average >= 80:
            return "B"
        elif average >= 70:
            return "C"
        elif average >= 60:
            return "D"
        else:
            return "F"

    def get_result(self):
        if self.calculate_average() >= 60:
```

```
        return "PASSED"
    return "FAILED"
```

Now:

```
student = Student("Krishna", 95, 88, 65)

print(f"Name: {student.name}")
print(f"Total: {student.calculate_total()}")
print(f"Average: {student.calculate_average():.2f}")
print(f"Grade: {student.calculate_grade()}")
print(f"Result: {student.get_result()}")
```

7. Multiple Objects

This is where OOP becomes really useful.

```
students = [
    Student("Krishna", 95, 88, 65),
    Student("Rahul", 82, 91, 78),
    Student("Priya", 90, 94, 89)
]
```

Now:

```
for student in students:
    print(f"Name: {student.name}")
    print(f"Average: {student.calculate_average():.2f}")
    print(f"Grade: {student.calculate_grade()}")
    print(f"Result: {student.get_result()}")
    print("-----")
```

You're now combining:

classes + objects + functions/methods + lists + loops + conditions

8. Inheritance

You already know this from Java.

Python:

```
class Person:

    def __init__(self, name):
        self.name = name

    def introduce(self):
        print(f"My name is {self.name}.")
```

Now:

```
class Student(Person):

    def study(self):
        print(f"{self.name} is studying.")
```

Create:

```
student = Student("Krishna")

student.introduce()
student.study()
```

Output:

```
My name is Krishna.
Krishna is studying.
```

Student inherited:

```
introduce()
```

from:

Person

9. super()

Java:

```
super();
```

Python:

```
super().__init__(name)
```

Example:

```
class Person:

    def __init__(self, name):
        self.name = name

class Student(Person):

    def __init__(self, name, program):
        super().__init__(name)
        self.program = program
```

Now:

```
student = Student("Krishna", "AIM2")

print(student.name)
print(student.program)
```

Output:

```
Krishna
AIM2
```

9. super()

Java:

```
super();
```

Python:

```
super().__init__(name)
```

Example:

```
class Person:

    def __init__(self, name):
        self.name = name

class Student(Person):

    def __init__(self, name, program):
        super().__init__(name)
        self.program = program
```

Now:

```
student = Student("Krishna", "AIM2")

print(student.name)
print(student.program)
```

Output:

```
Krishna
AIM2
```

So the Java → Python connection is:

Java	Python
this	self
Constructor	<code>__init__()</code>
extends	<code>class Child(Parent)</code>
<code>super()</code>	<code>super()</code>
Method	Method
Object	Object
Class	Class

10. One important Python difference from Java

In Java, you might write:

```
private String name;
```

Python doesn't use Java-style private fields in the same way.

You'll commonly see:

```
self.name
```

or, when **stronger name-mangling** is desired:

```
self.__name
```

DAY 7 EXERCISES

Now it's your turn. Don't copy the complete project yet. 😊

Exercise 1 — Basic Class

Create a class called:

Car

It should have:

brand

model

year

Create two objects.

For example:

Toyota Corolla 2022

Honda Civic 2024

Print their information.

Exercise 2 — Student Class

Create:

```
class Student:
```

The student should contain:

name

age

program

Create:

Krishna

28

AIM2

Add a method:

```
introduce()
```

Expected output:

```
Hello, my name is Krishna.
```

```
I am 28 years old.
```

```
I am studying AIM2.
```

★ Exercise 3 — Student Grade Class

Now create:

```
class Student:
```

Attributes:

```
name
```

```
math
```

```
python
```

```
ai
```

Methods:

```
calculate_total()
```

```
calculate_average()
```

```
calculate_grade()
```

```
get_result()
```

Use:

```
Krishna
```

```
95
```

```
88
```

```
65
```

Expected:

```
Name: Krishna
```

```
Total: 248
```

```
Average: 82.67
```

Grade: B
Result: PASSED

★ ★ DAY 7 MINI PROJECT

Now combine everything you've learned during **Week 1**.

Student Management System

Create a:

Student

class.

Then create at least **5 students**.

Example:

Krishna

Rahul

Priya

Amit

Neha

Each student should have:

name

age

math

python

ai

Your program should:

1. Display every student

Name: Krishna

Age: 28

2. Calculate total

Total: 248

3. Calculate average

Average: 82.67

4. Calculate grade

Grade: B

5. Determine pass/fail

Result: PASSED

6. Find the highest scorer

Highest Scorer: Priya

Highest Average: 91.00

7. Find the lowest scorer

Lowest Scorer: Amit

Lowest Average: ...

Why we're doing this project

Look at everything you've learned:

Variables

↓

Strings / Numbers / Boolean

↓
Lists / Dictionaries
↓
if / elif / else
↓
for / while
↓
Functions
↓
File handling
↓
Exception handling
↓
Modules
↓
Classes / Objects
↓
Student Management System

That's a **very solid Python Week 1 foundation.**